TRENDVIBE ATELIER

# Enterprise E-Commerce Platform Technical Interview Master Guide

Comprehensive Technical Reference: Java Jakarta EE, Servlet MVC, JDBC Architecture, MySQL 8-Table Schema, Concurrency Locking & Mobile Performance

---

| | |
|---|---|
| **Candidate / Role:** | Full-Stack Java Enterprise Developer / E-Commerce Software Engineer |
| **Project Name:** | TrendVibe Atelier (Haute Couture & Beauty Platform) |
| **Core Stack:** | Java 17 / Jakarta EE 10, Apache Tomcat 10.1, JDBC, MySQL 8.0, HTML5/CSS3, JavaScript |
| **Database Schema:** | 8 Normalized Relational Tables (Users, Categories, Products, Sizes, Cart, CartItems, Orders, OrderItems) |
| **Security & Auth:** | Phone + OTP Two-Factor Authentication, Role-based AuthFilter, BCrypt / PreparedStatements |
| **Catalog Scope:** | 260+ Curated Products across 5 Categories with Batch Pagination & Dynamic Payment Gateway |

## Table of Contents:


• 1. Architectural Blueprint & MVC Design Pattern

• 2. Servlet Lifecycle, Thread-Safety & Container Model

• 3. Database Design: 8-Table Schema, 3NF & Indexing Strategy

• 4. JDBC Connection Pooling vs DriverManager

• 5. Concurrency Control: Inventory Race Conditions & Flash Sales

• 6. Authentication, Session Security & Phone+OTP Gateway

• 7. 35 Frequently Asked Technical Interview Questions & Detailed Answers

# 1. Architectural Blueprint & MVC Design Pattern

TrendVibe Atelier is structured using a strict **Model-View-Controller (MVC)** enterprise architecture deployed on Apache Tomcat 10+ using the Jakarta EE specification. This separation ensures loose coupling, single responsibility, and high maintainability.

| Layer | Components & Packages | Key Responsibilities |
|---|---|---|
| **View Layer** | JSP, JSTL Core Tags, Vanilla CSS3, JS | Renders catalog, responsive mobile navigation, interactive payment mockup, order tracking timeline. |
| **Controller Layer** | com.ecommerce.controller.* (ProductServlet, CartServlet, CheckoutServlet, AuthServlet) | Intercepts HTTP GET/POST, handles routing, parses input parameters, coordinates with DAOs, dispatches to views. |
| **Model Layer** | com.ecommerce.model.* (User, Product, ProductSize, Order, CartItem) | Plain Old Java Objects (POJOs) representing domain entities with encapsulation and serialization. |
| **DAO / Data Access** | com.ecommerce.dao.* (ProductDAO, UserDAO, OrderDAO, CartDAO, DBConnection) | Encapsulates JDBC logic, executes parameter-bound PreparedStatements, maps ResultSets to POJOs. |
| **Database** | MySQL 8.0+ (8 Normalized Tables) | ACID transactional persistence, foreign key cascading, composite indexes on category/pincode. |

# 2. Servlet Lifecycle & Thread-Safety

A critical interview topic is how the servlet container (Tomcat) manages servlet lifecycles and threads:

**1. Class Loading & Instantiation:** When Tomcat starts or on first request (load-on-startup), the container loads the servlet class and instantiates a *single* instance.
**2. Initialization:** The container calls init(ServletConfig config) once. Used for loading DB configs and caches.
**3. Request Handling:** For each incoming HTTP request, Tomcat allocates a thread from its thread pool and invokes service(HttpServletRequest, HttpServletResponse), which delegates to doGet() or doPost().
**4. Destruction:** When Tomcat shuts down or the app undeploys, destroy() is invoked once to release resources.

### Thread-Safety Warning in Servlets:

Because **only one instance** of each Servlet exists in memory and serves multiple concurrent threads, **instance variables are shared across all requests**! Declaring stateful instance variables in a Servlet causes severe race conditions. In TrendVibe Atelier, all request state is confined strictly to method-local variables.

# 3. Database Schema: 8 Normalized Relational Tables

The schema strictly adheres to **Third Normal Form (3NF)**, eliminating data redundancy and update anomalies:

| Table | Primary Key | Foreign Keys | Key Columns & Notes |
|---|---|---|---|
| users | user_id | None | phone (UNIQUE, 10-digit), email, full_name, pincode |
| categories | category_id | None | category_name (Men, Women, Beauty, Footwear, Watches) |
| products | product_id | category_id | product_name, price, mrp, discount_percent, image_url |
| product_sizes | size_id | product_id | size_label (S, M, L, XL), stock_quantity, sku_code |
| cart | cart_id | user_id | Active user bag container, timestamps |
| cart_items | cart_item_id | cart_id, product_id | selected_size, quantity, unit_price |
| orders | order_id | user_id | order_number (#TV-92041), total_amount, status, awb_code |
| order_items | item_id | order_id, product_id | Snapshot of price, quantity, size at order placement time |

# 4. JDBC Connection Pooling vs DriverManager

**Why not use DriverManager.getConnection() in production?**
Each call to DriverManager.getConnection() initiates a TCP 3-way handshake with MySQL, performs TLS negotiation, and allocates database backend processes. This incurs 50–100ms of latency per query and crashes under 100+ concurrent requests.
**Connection Pooling Solution (HikariCP / Tomcat DataSource):**
Maintains a pre-warmed pool of active physical connections (e.g. 10–50 connections). When a DAO requests a connection via dataSource.getConnection(), a pre-existing connection is borrowed in microseconds. Calling conn.close() returns the connection to the pool rather than tearing down the socket.

# 5. Concurrency Control: Inventory Locking in Flash Sales

During high-concurrency events (e.g. festive flash sales where only 2 items remain in size 'M'), multiple users will click 'Pay' simultaneously. This creates a classic *check-then-act race condition*.

## A. Pessimistic Locking (SELECT ... FOR UPDATE):

```
Locks the specific product_size row at the database engine level until transaction commit:
START TRANSACTION;
SELECT stock_quantity FROM product_sizes WHERE size_id = 42 FOR UPDATE;
-- If stock_quantity >= order_qty:
UPDATE product_sizes SET stock_quantity = stock_quantity - 1 WHERE size_id = 42;
INSERT INTO orders ...;
COMMIT;
```

## B. Optimistic Locking with Atomic UPDATE (Recommended):

```
Avoids heavy database lock queues by utilizing atomic conditional SQL updates:
UPDATE product_sizes SET stock_quantity = stock_quantity - 1
WHERE size_id = 42 AND stock_quantity >= 1;
If executeUpdate() returns 0, the inventory is sold out, and the application immediately aborts
the payment gracefully without table locks.
```

# 6. Top 35 Technical Interview Questions & Model Answers

### Q1: What is the primary difference between Servlet filters and listeners?

A Filter intercepts requests and responses dynamically before reaching servlets (e.g., AuthFilter for authentication, gzip compression, logging). A Listener responds to container lifecycle events (e.g., HttpSessionListener tracks active users, ServletContextListener initializes connection pools).

### Q2: Explain why PreparedStatements prevent SQL Injection.

PreparedStatements pre-compile the SQL structure in the database engine first with placeholders (?). When parameter values are bound (e.g., setString(1, input)), the database treats them strictly as literal data strings, never as executable SQL instructions, rendering injection payloads harmless.

### Q3: How is ACID guaranteed during the checkout process?

Atomicity: Either both order creation and stock decrement succeed, or all rollback via conn.rollback(). Consistency: Foreign keys and stock >= 0 checks are enforced. Isolation: Read Committed or Repeatable Read isolation levels prevent dirty reads. Durability: MySQL InnoDB write-ahead redo log ensures committed orders survive crashes.

### Q4: What is the difference between RequestDispatcher.forward() and HttpServletResponse.sendRedirect()?

forward() executes entirely server-side within the container. The browser URL does not change, request attributes are preserved, and only 1 roundtrip occurs. sendRedirect() sends an HTTP 302 back to the browser with a new Location header, causing the browser to initiate a new GET request with a new request object.

### Q5: How does the session mechanism work between browser and Tomcat?

When request.getSession() is called, Tomcat creates an in-memory session object and returns a cookie named JSESSIONID with an HttpOnly flag. On subsequent requests, the browser transmits JSESSIONID in request headers, allowing Tomcat to map the request to the corresponding HttpSession.

### Q6: How would you scale session state in a multi-node Tomcat cluster?

By default, sessions are stored in Tomcat's local JVM memory, failing behind a round-robin load balancer. Solutions include: (1) Sticky sessions via Nginx/HAProxy (least resilient), (2) Distributed session cache using Redis or Memcached via Redisson/Spring Session, or (3) Migrating to stateless JWT authentication.

### Q7: What is the role of web.xml in Jakarta EE 10?

web.xml is the Deployment Descriptor. It configures servlet mappings, filter chains, session timeout parameters, error-page redirects, and context-params. In modern Jakarta EE, annotations like @WebServlet and @WebFilter can replace web.xml, but web.xml retains overriding precedence.

### Q8: How does connection pooling improve performance over raw JDBC?

Opening physical sockets and handshaking with MySQL takes ~80ms per query. Connection pools (HikariCP) pre-establish pooled TCP connections and borrow them in sub-milliseconds (<0.5ms), managing maxLifetime, idleTimeout, and connection leak detection.

### Q9: What happens if a Servlet class has a non-thread-safe instance variable?

Because Tomcat uses a multi-threaded single-instance model, concurrent threads executing doGet()/doPost() will read and overwrite the shared instance variable simultaneously, causing data corruption, phantom values, and critical user session mixing.

### Q10: What is the N+1 query problem and how did you avoid it in TrendVibe?

N+1 occurs when fetching N orders, and then executing 1 separate SQL query per order to fetch its items (N+1 database roundtrips). In TrendVibe, we use SQL JOIN queries (SELECT * FROM orders o JOIN order_items oi ON o.order_id = oi.order_id) to fetch all data in a single efficient query.

### Q11: Explain the purpose of JSTL (JSP Standard Tag Library).

JSTL replaces messy Java scriptlet tags (<% ... %>) with clean, declarative XML tags (, , ). This prevents scriptlet runtime exceptions and strictly decouples business logic from view presentation.

### Q12: How is the Phone + OTP authentication flow verified securely?

The client sends a 10-digit mobile number. The server generates a cryptographically secure 4-digit code (SecureRandom), stores it in a cache with a 10-minute TTL and max-attempt threshold (3 tries), and dispatches via SMS gateway. Upon submission, the server verifies the hash and clears the OTP token immediately.

### Q13: How do you prevent Double-Click / Double-Submit order duplicate charges?

Client-side: Immediately disable the 'Authorize Payment' button upon first click and display a loading spinner. Server-side: Generate an Idempotency Key (UUID) attached to the checkout session. If a second request arrives with the same key, return the existing order rather than creating a duplicate.

### Q14: Explain Database Normalization and why 3NF was chosen.

1NF removes repeating groups (atomic columns). 2NF removes partial key dependencies. 3NF removes transitive dependencies (non-key attributes depending on other non-key attributes). 3NF ensures product categories or sizes can be updated without touching orders, preserving historical billing integrity.

### Q15: How are product images loaded efficiently across 260+ items on mobile devices?

Using three techniques: (1) Dynamic batch pagination (loads 24 items initially, keeping DOM lean), (2) Native lazy loading (loading='lazy') so off-screen images are only requested when scrolling near, and (3) CDN dimension cropping (auto=format&fit;=crop&w;=800&q;=80).

### Q16: How do you handle database deadlocks in MySQL InnoDB?

Deadlocks occur when transaction A locks row 1 and waits for row 2, while transaction B locks row 2 and waits for row 1. InnoDB automatically detects deadlocks and rolls back the smaller transaction. The application catches SQLException (error code 1213) and retries the transaction with exponential backoff.

### Q17: What is the difference between executeQuery(), executeUpdate(), and execute()?

executeQuery() is used for SELECT queries and returns a ResultSet. executeUpdate() is used for INSERT, UPDATE, DELETE, or DDL statements and returns an int representing affected rows. execute() is used when the SQL type is dynamic or returns multiple result sets.

### Q18: What is the front controller design pattern?

A single centralized controller servlet intercepts all incoming web requests, enforces uniform security, logging, and locale resolution, and delegates to specialized action handlers. Spring's DispatcherServlet is a classic implementation of this pattern.

### Q19: How do you implement Cross-Site Request Forgery (CSRF) protection?

By generating a random, unpredictable anti-CSRF token per user session, injecting it into forms as a hidden field, and verifying that incoming POST requests contain the matching token in AuthFilter before proceeding.

### Q20: What are the key differences between Java EE 8 and Jakarta EE 10?

Oracle transitioned Java EE to the Eclipse Foundation, requiring a package namespace change from javax.* (e.g. javax.servlet) to jakarta.* (e.g. jakarta.servlet). Jakarta EE 10 requires Java 11/17+ and Apache Tomcat 10.1+.

### Q21: How does the browser enforce HttpOnly and Secure cookie flags?

HttpOnly blocks client-side JavaScript (document.cookie) from reading the session cookie, neutralizing XSS credential theft. Secure instructs the browser to only transmit the cookie over encrypted HTTPS connections.

### Q22: Why should money and prices be handled with BigDecimal or integer cents rather than double?

double uses IEEE 754 floating-point arithmetic which causes binary rounding errors (e.g. 0.1 + 0.2 = 0.30000000000000004). Financial calculations require exact precision, provided by java.math.BigDecimal or integer currency units (cents/paise).

### Q23: How would you implement search autocomplete across 260+ products efficiently?

On client-side: Use debounce (300ms) to avoid firing requests on every keystroke. On server-side: Use SQL prefix indexes (WHERE product_name LIKE 'linen%') or full-text search indexes (MATCH(product_name, brand) AGAINST(...)).

### Q24: What is the DAO (Data Access Object) design pattern?

DAO abstracts and encapsulates all interactions with the persistence layer. The rest of the application interacts with clean Java interfaces without knowing whether data is backed by MySQL, PostgreSQL, or an external REST API.

### Q25: What is the difference between optimistic locking and pessimistic locking?

Pessimistic locking assumes conflict will happen and places exclusive row locks immediately (blocking other readers/writers). Optimistic locking assumes conflict is rare, verifies a version column at update time, and retries if another process updated the record first.